

Nombre de la Universidad

Facultad de Ciencias

---

# Título de la Tesis

---

Tesis presentada para obtener el grado de  
Nombre del Grado Académico

Presentado por:

Tu Nombre

Director de Tesis:

Nombre del Director

Ciudad, País

30 de enero de 2025

# Índice

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                  | <b>2</b>  |
| <b>2. Gráficas y Superficies</b>                                        | <b>2</b>  |
| 2.1. Definiciones y conceptos . . . . .                                 | 2         |
| 2.2. Matriz de adyacencia . . . . .                                     | 5         |
| 2.3. Grado de un vértice . . . . .                                      | 5         |
| 2.4. Subgráficas . . . . .                                              | 7         |
| 2.5. Caminos y ciclos . . . . .                                         | 8         |
| 2.6. Árboles . . . . .                                                  | 9         |
| <b>3. Redes Neuronales Gráficas</b>                                     | <b>12</b> |
| 3.1. Redes Neuronales . . . . .                                         | 12        |
| 3.2. Redes convolucionales CNN . . . . .                                | 18        |
| 3.3. Redes Neuronales Gráficas . . . . .                                | 22        |
| 3.4. Propagación de Mensajes Neuronales . . . . .                       | 22        |
| 3.5. Descripción general del mecanismo . . . . .                        | 23        |
| 3.6. La GNN fundamental . . . . .                                       | 23        |
| 3.7. Redes Gráficas Convolucionales (GCNs) . . . . .                    | 24        |
| <b>4. Muestreo General</b>                                              | <b>25</b> |
| 4.1. Superficies estudiadas . . . . .                                   | 25        |
| 4.2. Métodos de Muestreo . . . . .                                      | 26        |
| 4.3. Orden por árbol generador mínimo . . . . .                         | 26        |
| 4.4. Muestreo por Esferas . . . . .                                     | 27        |
| 4.5. Reasignación de Adyacencias para Preservar la Estructura . . . . . | 32        |
| 4.6. Método general para la reducción de nodos . . . . .                | 34        |
| 4.6.1. Combinación de funciones . . . . .                               | 34        |
| 4.7. Análisis de Complejidad . . . . .                                  | 35        |
| <b>5. Resultados y conclusiones</b>                                     | <b>36</b> |
| 5.1. Arquitectura PointNet . . . . .                                    | 37        |
| 5.2. Resultados de la evaluación . . . . .                              | 38        |
| 5.3. Conclusiones . . . . .                                             | 38        |

## 1. Introducción

## 2. Gráficas y Superficies

La teoría de gráficas y el estudio de superficies son dos ramas de las matemáticas que en primera instancia podrían parecer ajenas entre sí. Sin embargo a lo largo de este capítulo veremos que ambas están relacionadas a través de la implementación de triangulaciones sobre las superficies. En particular la representación de superficies a través de gráficas proporciona una nueva perspectiva que permite estudiar propiedades topológicas de otros objetos matemáticos conocidos como nudos.

Una gráfica es un conjunto de vértices que están conectados por aristas, mientras que una superficie puede entenderse como un objeto que es localmente homeomorfo al plano  $\mathbb{R}^2$ , tales como una esfera, o un toro. Al triangular la superficie, podemos relacionarla posteriormente con una gráfica, lo cual permite la posibilidad de abordar la superficie con los teoremas y algoritmos existentes para la teoría de gráficas. Esta relación también permite investigar temas como la clasificación de superficies.

Este capítulo explora la relación entre ambas áreas, enfocándose en como la triangulación de superficies nos proporciona una herramienta para la comprensión de propiedades topológicas básicas, así como algoritmos utilizados para el estudio de dichas áreas. La gran mayoría de las definiciones, conceptos y algoritmos que se estudian son basadas en los textos: [4], [2], [6], las cuales utilizaremos para capítulos posteriores.

### 2.1. Definiciones y conceptos

Para entender el objeto de estudio principal de esta tesis, comenzaremos por definir uno de los objetos matemáticos que utilizaremos en primera instancia, el concepto de superficie.

**Definición 1.** (*Superficie*)

Un subconjunto  $S$  de  $\mathbb{R}^3$  es una **superficie** si para cada punto  $p \in S$ , hay un conjunto abierto  $U \subset \mathbb{R}^2$  y un conjunto abierto  $W \subset \mathbb{R}^3$  que contiene a  $p$  tal que  $S \cap W$  es homeomorfo a  $U$ .

**Ejemplo 1.** Sea  $S = \{(x, y, z) \in \mathbb{R}^3 | z = x^2 - y^2\}$ . Demostraremos que  $S$  es una superficie, entonces, tenemos que demostrar que para cada punto  $p \in S$  existen conjuntos,  $U \in \mathbb{R}^2$ ,  $W \in \mathbb{R}^3$  abiertos tal que  $p \in W \cap S$  y que existe  $h : S \cap W \subset \mathbb{R}^3 \rightarrow U \subset \mathbb{R}^2$  tal que  $h$  es un homeomorfismo.

*Demostración.* Sea  $p = (x_0, y_0, z_0) \in S$  entonces por definición  $z_0 = x_0^2 - y_0^2$ , por otro lado definimos a  $W \subset \mathbb{R}^3$  como la bola abierta centrada en  $p$  de radio  $r$   $\mathcal{B}_p(r)$  y a  $U \subset \mathbb{R}^2$  como la vecindad alrededor de  $(x_0, y_0)$ ,  $V(x_0, y_0)$ .

Definamos ahora la función  $\Pi : S \cap W \rightarrow U$  dada por:

$$\Pi(x, y, x) = (x, y)$$

Es decir la función proyección restringida a  $S \cap W$ , ahora notemos lo siguiente:

- $\Pi$  es una función continua ya que es la restricción de la función proyección que es continua en  $\mathbb{R}^3$ .

- $\Pi$  es inyectiva pues, dado  $u = (x_1, y_1, z_1), w = (x_2, y_2, z_2) \in S \cap W$  tenemos que :

$$\begin{aligned}\Pi(x_1, y_1, z_1) = \Pi(x_2, y_2, z_2) &\Rightarrow \\ (x_1, y_1) = (x_2, y_2) &\Rightarrow \\ x_1 = x_2, y_1 = y_2 &\end{aligned}$$

Por otro lado como  $u, w \in S$  entonces:

$$z_1 = x_1^2 - y_1^2 = x_2^2 - y_2^2 = z_2$$

Es decir :

$$u = (x_1, y_1, z_1) = (x_2, y_2, z_2) = w$$

Por tanto  $\Pi$  es inyectiva.

- Sea  $(x, y) \in U$  entonces notamos que existe un punto en  $p \in S \cap W$  dado por  $p = (x, y, x^2 - y^2)$  por tanto  $\Pi$  es sobreyectiva.
- Notamos que  $\Pi^{-1}$  está dada por  $\pi^{-1}(x, y) = (x, y, x^2 - y^2)$ , como las componentes  $x, y$  son continuas y  $x^2 - y^2$  es una combinación lineal de componentes continuas entonces  $\Pi^{-1}(x, y) = (x, y, x^2 - y^2)$  es continua

Por tanto  $\Pi$  es un homeomorfismo y por tanto  $S \cap W$  es homeomorfo a  $U$  y por lo tanto  $S$  es una superficie.  $\square$

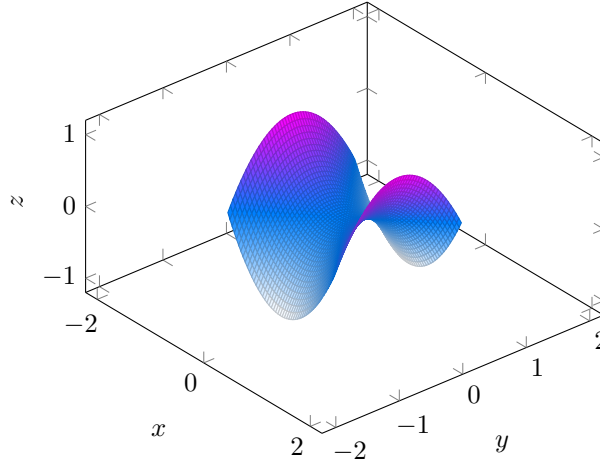


Figura 1: Ejemplo de superficie

En la figura 1 se puede observar una superficies en  $\mathbb{R}^3$  comúnmente conocida como "Silla de montar"

Habiendo establecido una forma rigurosa de definir superficies, es útil explorar cómo podemos representar estas superficies mediante estructuras más manejables y adecuadas para los objetivos de esta tesis. Una técnica clave es la triangulación, que nos permite descomponer

una superficie en elementos simples, como triángulos, preservando su estructura topológica. En este contexto, el teorema de triangulación de Radó desempeña un papel central, ya que garantiza que cualquier superficie puede ser triangulada. Este proceso nos permite asociar una gráfica a la superficie al considerar los vértices y aristas de la triangulación, facilitando su estudio como gráfica.

**Definición 2.** (*Triangulación*)

Una triangulación de una superficie  $S$  es una subdivisión de  $S$  en triángulos topológicos que se intersectan en aristas o en vértices.

**Teorema 1.** (*Teorema de Radó*)

Toda superficie  $S$  se puede triangular

**Definición 3** (Gráfica). Una gráfica simple  $G$  está definida por un par ordenado  $G = (V, E)$ , donde  $V$  es un conjunto cuyos elementos son llamados vértices (o nodos) de la gráfica y  $E$  un conjunto cuyos elementos son las aristas de la gráfica. Regularmente denotamos a  $V$  y  $E$  como  $V(G)$  y  $E(G)$  que hacen referencia a la gráfica  $G$ , entonces  $G = (V(G), E(G))$  Definimos a  $E$  como:

$$E(V) = \{\{u, v\} | u, v \in V, u \neq v\}$$

Usualmente denotamos al par  $\{u, v\}$  simplemente como  $uv$ .

**Definición 4** (Orden y tamaño). Para una gráfica  $G$ , tenemos que:

1.  $\nu_G = |V(G)|$  es llamado el orden de  $G$ .
2.  $\varepsilon_G = |E(G)|$  es llamado el tamaño de  $G$ .
3. Para cada arista  $e = uv \in E$  los vértices  $u$  y  $v$  son llamados extremos de la arista.
4. Los vértices  $u$  y  $v$  son adyacentes o vecinos si  $e = uv \in E$ .
5. Dos aristas  $e_1 = uv$  y  $e_2 = uw$  tienen un extremo común, entonces son adyacentes entre sí.

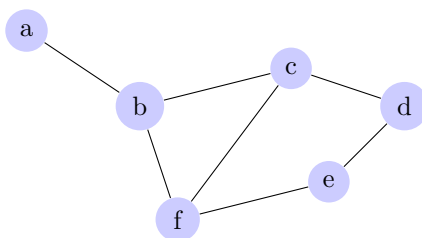


Figura 2: Ejemplo de gráfica simple  $G$

En la figura 2 podemos observar que  $\nu_G = 6$ ,  $\varepsilon_G = 7$ , los extremos de la arista  $ab$  son los vértices  $a$  y  $b$ , también los vértices  $a$  y  $b$  son vecinos y por último las aristas  $ab$  y  $bf$  son adyacentes ya que comparten el vértice  $b$ .

## 2.2. Matriz de adyacencia

Sabemos que las figuras planas mostradas en la sección anterior, son fácilmente comprensibles para nuestros ojos, pero al momento de crear algoritmos o de computarlas es muy costoso en términos de memoria por lo cual, es necesario el manejo de estos objetos de una forma un poco más comprensible para cualquier computadora, de esta forma nace la **matriz de adyacencia** que definiremos a continuación.

**Definición 5** (Matriz de Adyacencia). Sea  $V_G = \{v_1, \dots, v_n\}$  ordenado. La matriz de adyacencia  $\mathbf{A}$  de  $\mathbf{G}$  está definida como

$$\mathbf{A}(\mathbf{G}) = [a_{ij}]$$

donde;

$$a_{ij} = \begin{cases} 1 & \text{si } v_i v_j \in \mathbf{E}_G \\ 0 & \text{si no} \end{cases}$$

La matriz de adyacencia de la figura 2 quedaría de la siguiente forma:

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (1)$$

## 2.3. Grado de un vértice

**Definición 6** (Vecindario). Sea  $v \in \mathbf{G}$  un vértice de la gráfica  $\mathbf{G}$ . El **vecindario** de  $v$  es el conjunto dado por:

$$N_G(v) = \{u \in G \mid vu \in \mathbf{E}(\mathbf{G})\}$$

**Definición 7** (Grado de un vértice). El **grado** de  $v$  es el número de sus vecinos.

$$d_G(v) = |N_G(v)|.$$

De la definición anterior podemos notar que para cualquier vértice  $v$ , podría suceder que  $v$  no tenga vecinos o por el otro lado que el resto de los vértices de  $G$  sean sus vecinos, recordando que  $v$  no puede ser vecino de sí mismo. Así, se tiene que

$$0 \leq d_G(v) \leq n - 1$$

donde  $n$  es el orden la gráfica  $G$ .

**Definición 8** (Vértice aislado). Cuando  $d_G(v) = 0$  decimos que  $v$  es un vértice aislado.

Dada una gráfica  $G = (V, E)$  con  $V(G) = \{v_1, \dots, v_n\}$  sabemos que el conjunto:

$$\{d_G(v_1), \dots, d_G(v_n)\}$$

Es un conjunto finito de números naturales pues el conjunto  $V(G)$  lo es, entonces de esta forma podemos definir a  $\delta(G)$  como el grado mínimo de  $G$  y  $\Delta(G)$  como el grado máximo de  $G$ .

**Definición 9** (Grado mínimo y grado máximo). *Sea  $G = (V, E)$  una gráfica simple, tenemos que;*

1.  $\delta(G) = \min\{d_G(v_1), \dots, d_G(v_n)\}$  es el grado mínimo de  $G$ .
2.  $\Delta(G) = \max\{d_G(v_1), \dots, d_G(v_n)\}$  es el grado máximo de  $G$ .

Para la gráfica de la figura 2 tenemos que  $\delta(G) = 1$  y que  $\Delta(G) = 3$ .

**Lema 1. (*Lema del saludo de manos*)** Para cada gráfica  $G$  tenemos que:

$$\sum_{v \in V(G)} d_G(v) = 2 \cdot \varepsilon_G$$

Además, el número de vértices de grado impar es par.

*Demostración.*

□

**Definición 10** (Grado promedio). *Dada una gráfica  $G$ , definimos el **grado promedio** de  $G$  como:*

$$d(G) = \frac{1}{\nu_G} \sum_{v \in V(G)} d_G(v)$$

De ahí se reduce inmediatamente por **lema 5** que

$$d(G) = 2 \frac{\varepsilon_G}{\nu_G}$$

**Proposición 1.** *Dada una gráfica  $G$ , tenemos que*

$$\delta(G) \leq d(G) \leq \Delta(G).$$

*Demostración.* Sabemos que;

$$\nu_G \cdot \delta(G) \leq \sum_{v \in V(G)} d_G(v)$$

Pues

$$\delta(G) \leq d_G(v_i) \quad \forall i \in \{1, \dots, n\}$$

Como  $0 \leq \frac{1}{\nu_G}$  entonces ;

$$\delta(G) \leq \frac{1}{\nu_G} \sum_{v \in V(G)} d_G(v)$$

Por tanto

$$\delta(G) \leq d(G).$$

Por otro lado cómo

$$d_G(v_i) \leq \Delta(G) \quad \forall i \in \{1, \dots, n\}$$

Entonces;

$$\sum_{v \in V(G)} d_G(v) \leq \nu_G \cdot \Delta(G)$$

Y por lo observado arriba;

$$\frac{1}{\nu_G} \sum_{v \in V(G)} d_G(v) \leq \Delta(G)$$

Así;

$$d(G) \leq \Delta(G)$$

$$\therefore \delta(G) \leq d(G) \leq \Delta(G)$$

□

## 2.4. Subgráficas

Debido a los métodos que se utilizarán más adelante en el estudio de los objetos y debido al gran número de datos a procesar, es necesario introducir conceptos que se púeden emplear para el estudio local de las gráficas. De este modo introduciremos el concepto de subgráfica.

**Definición 11** (Subgráfica). *Una gráfica  $H$  es subgráfica de una gráfica  $G$  denotado por  $H \subseteq G$ , si  $V(H) \subseteq V(G)$  Y  $E(H) \subseteq E(G)$ .*

**Definición 12.** *Decimos que una subgráfica  $H \subseteq G$  abarca  $G$  (y  $H$  es una subgráfica generadora de  $G$ ) si cada vértice de  $G$  está en  $H$ , i.e  $V(H) = V(G)$*



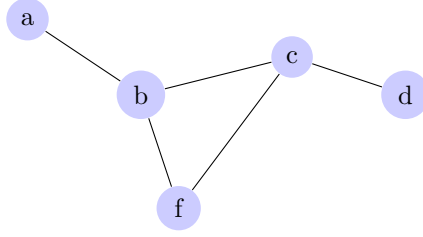


Figura 3: Ejemplo de subgráfica de G

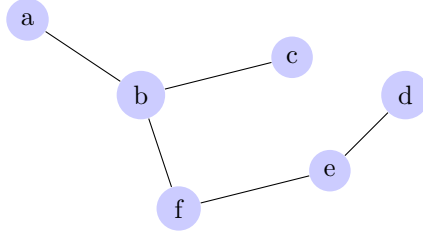


Figura 4: Ejemplo de una gráfica H que abarca G

**Definición 13.** Decimos que una subgráfica  $H \subseteq G$ , es una subgráfica inducida, si  $E(H) = E(G) \cap E(V_H)$ . En este caso  $H$  es inducido por el conjunto  $V(H)$  de vértices.

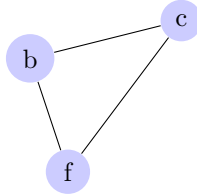


Figura 5: Ejemplo de una gráfica inducida

## 2.5. Caminos y ciclos

Uno de los conceptos fundamentales que se introducirán y además se implmentarán en la creación de los algoritmos de la sección final es el concepto de caminos, este concepto se utilizará cómo herramienta para el estudio y recorrido de la gráfica completa.

**Definición 14** (Camino, ciclo y caminos cerrados). Sea  $e_i = u_i u_{i+1} \in E(G)$  aristas de  $G$  para  $i \in \{1, \dots, k\}$ .

Un camino es una gráfica no vacía  $W = (V, E)$  de la forma;

$$V = \{u_0, u_1, \dots, u_k, u_{k+1}\} \quad E = \{e_0, e_1, \dots, e_k\}$$

donde:  $u_i$  es adyacente a  $u_{i+1}$  para todo  $i \in \{1, \dots, k\}$ .

Además:

1.  $W$  es **cerrado**, si  $u_0 = u_{k+1}$ .
2.  $W$  es un **camino**, si  $u_i \neq u_j$ , para todo  $i \neq j$ .

3.  $W$  es un **ciclo**, si es cerrado y  $u_i \neq u_j$  para  $i \neq j$  excepto para  $u_1 = u_{k+1}$

Además, el número  $l(W)$  es el tamaño o longitud del camino y está dado por  $l(W) = |E(W)|$

## 2.6. Árboles

Una forma de reducir el número de vértices sin que la gráfica pierda gran parte de su información, es mediante el estudio de los árboles, veremos su definición y los conceptos más importantes relacionados con el estudio de estas estructuras.

**Definición 15** (Gráfica conexa). Una gráfica  $G = (V, E)$  es conexa si para cualquier par de vértices  $u$  y  $v$  de  $G$  si existe un camino que inicia en  $u$  y termina en  $v$

**Definición 16** (Componente conexo). Sea  $G = (V, E)$  una gráfica. Un **componente conexo** de  $G$  es una subgráfica  $H$  tal que:

1.  $H$  es conexa.
2.  $H$  es maximal con respecto a la propiedad de ser conexa. Es decir, no existe una subgráfica conexa más grande que contenga a  $H$ .

**Definición 17** (Punto de articulación). Dada un gráfica  $G$  conexa y una arista  $e \in E_G$ . Decimos que  $e$  es un puente de la gráfica  $G$ , si  $G - e$  es desconexa.

Si tomamos como ejemplo la gráfica de la figura 2 notamos que el vértice  $e = ab$  es un puente de  $G$ . Pues la gráfica  $G - e$  queda en dos componentes, la conformada por el vértice  $\{a\}$  y la que contiene a los vértices  $\{b, c, d, e, f\}$ .

**Proposición 2.** Sea una gráfica  $G$ . Un vértice  $e \in E(G)$  es un puente de  $G$  si y sólo si  $e$  no está en ningún ciclo de  $G$ .

*Demostración.* Primero notemos que  $e = uv$  es un puente si y sólo si  $u, v$  pertenecen a diferentes componentes conexas de  $G - e$ , pues de lo contrario implicaría que existe un camino que conecta a  $u$  con  $v$  lo cual sería una contradicción ya que  $G - e$  es desconexo.

( $\Rightarrow$ ) Por contrapositiva, Supongamos que existe un ciclo de  $G$  que contiene a  $e$ , entonces existe un camino tal que  $C = \{u_0 = u, v, \dots, u_{k+1} = u\}$ . De aquí, notamos que en  $G - e$  existe un camino  $C' = \{v_0 = v, \dots, v_{k+1} = u\}$ , entonces existe un camino alternativo que conecta a  $u$  con  $v$  lo cual implica que  $G - e$  es conexa. Por tanto  $e = uv$  no es puente.

( $\Leftarrow$ ) Supongamos que  $e = uv$  no es puente, por la observación hecha al inicio esto implica que  $u, v$  pertenecen a la misma componente conexa de  $G - e$ . Entonces existe un camino que  $C = \{v_0 = v, \dots, v_{k+1} = u\}$  conecta a  $v$  con  $u$ , de ahí notamos que  $eC$  es un ciclo pues  $eC = \{u, v_0 = v, \dots, v_{k+1} = u\}$ .  $\square$

**Definición 18** (Gráfica acíclica). Una gráfica es llamada **acíclica** si no contiene ciclos.

**Definición 19** (Árbol). Un **árbol** es una gráfica acíclica y conexa.

De la **Proposición 2.2** y la **Definición 22** se deriva el siguiente corolario.

**Corolario 1.** Una gráfica conexa es un árbol si y sólo si todas sus aristas son puentes.

**Teorema 2.** *Los siguientes enunciados son equivalentes:*

1.  $T$  es un árbol
2. Cualesquiera dos vértices están conectados en  $T$  por un único camino.
3.  $T$  es acíclico y  $\varepsilon_T = \nu_T - 1$

*Demostración.* Sea  $\nu_T = n$ , Si  $n=1$  el teorema es trivial, así que supongamos para  $n \geq 2$ .

(1)  $\Rightarrow$  (2) Supongamos que  $T$  es un árbol. Sean  $u$  y  $v$  dos vértices de  $T$  y por contradicción supongamos que existen dos caminos  $P$  y  $Q$  tales que conectan a  $u$  y  $v$ . Sin pérdida de generalidad supongamos que  $l(P) > l(Q)$ . Entonces existe al menos una arista  $e$  que está en  $P$  pero no está en  $Q$ . Por **Corolario 2** sabemos que cada arista de  $T$  es un puente. Y por lo visto anteriormente entonces  $u$  y  $v$  están en diferentes componentes conexas de  $T - e$ . Pero notamos que por el camino  $Q$  siguen estando conectados. Lo cual es una contradicción ya que implicaría que  $u$  y  $v$  no estén en diferentes componentes conexas.

(2)  $\Rightarrow$  (3) Esta implicación la probaremos por inducción sobre  $n$ .

Para  $n = 2$ . Sabemos que no existen componentes desconexas pues cualesquiera dos vértices están conectados por un camino, por lo cual  $\varepsilon_T = 1 = 2 - 1 = \nu_T - 1$ .

Supongamos que la afirmación se cumple para los valores menores a  $n$  y probemos para  $n$ . Sea  $P$  el máximo camino en  $T$  entre  $u$  y  $v$ , es decir que no existen aristas  $e$  tales que  $eP$  o  $Pe$  sean caminos. De ahí que  $d_T(v) = 1$ . Si  $uw \in E(T)$  entonces  $w \in P$ . Ahora notamos que la subgráfica  $T - v$  está conectada por un único camino  $P'$ , además notamos que  $l(P)$  es a lo más  $n$  y cómo  $l(P) > l(P')$  entonces  $l(P') \leq n - 1$  dado que la subgráfica  $T - v$  está conectada por un único camino (Pues  $T$  lo está) y por hipótesis de inducción tenemos que:

$$\varepsilon_{T-v} = \nu_{T-v} - 1 = (n - 1) - 1 = n - 2$$

Y de ahí que:

$$\varepsilon_T = \varepsilon_{T-v} + 1 = n - 1$$

(3)  $\Rightarrow$  (1) Supongamos (3). Basta con probar que  $T$  es conexo. Sean  $T_i = (V_i, E_i)$  componentes conexas de  $T$  para  $i \in \{1, \dots, k\}$ . Sabemos que  $T$  es acíclico, por lo tanto sus componentes conexas  $T_i$  también lo son por tanto  $|E_i| = |V_i| - 1$ . Notemos que:

$$\nu_T = \sum_{i=1}^k |V_i|, \varepsilon_T = \sum_{i=1}^k |E_i|$$

Entonces:

$$n - 1 = \varepsilon_T = \sum_{i=1}^k (|V_i| - 1) = \sum_{i=1}^k |V_i| - k = n - k$$

$$\therefore k = 1$$

Lo cual quiere decir que  $T$  es un sólo componente conexo, por lo tanto  $T$  es un árbol.  $\square$

**Definición 20** (Árbol generador). *Dada una gráfica conexa  $G$  una subgráfica  $T$  de  $G$  es un subárbol generado de  $G$  si satisface que;*

1.  $T$  es una subgráfica generadora de  $G$  y
2.  $T$  es un árbol.

**Teorema 3.** *Cada gráfica conexa contiene un árbol generador.*

*Demostración.* Sea  $H \subset G$  la mínima gráfica generadora conexa de  $G$ , es decir  $H - e$  es disconexa para todo  $e \in E(H)$ . Esta gráfica es obtenida de  $G$  removiendo todas las aristas que no son puentes:

1. Hacemos  $H_0 = G$ .
2. Para  $i \geq 0$  sea  $H_{i+1} = H_i - e_i$  donde  $e$  no es un puente de  $H_i$ . Como  $e_i$  no es un puente  $H_{i+1}$  es una subgráfica generadora y conexa de  $H_i$  y por lo tanto de  $G$ .
3. De este forma hacemos  $H = H_k$  cuando sólo quedan puentes en la subgráfica. Y por el **Corolario 2**  $H$  es un árbol.

□

**Definición 21** (Árbol generador mínimo). *Un árbol generador mínimo es un árbol generador tal que la suma de los pesos de sus aristas es la mínima posible, es decir.*

$$w(T) = \sum_{(u,v) \in T} \min\{w(u,v)\}$$

Donde  $w(u,v)$  es el peso que hay entre el vértice  $u$  y la arista  $v$ .

**Definición 22** (Característica de Euler). *Dado un grafo plano  $G$ , la característica de Euler se define como:*

$$\chi(G) = V - E + F$$

*Y esta propiedad es un **invariante topológico**.*

### 3. Redes Neuronales Gráficas

En las últimas décadas, las redes neuronales han emergido como una herramienta fundamental en el campo del aprendizaje automático y la inteligencia artificial. Su capacidad para aprender de grandes volúmenes de datos, reconocer patrones complejos y adaptarse a distintos entornos ha sido clave para su popularidad y adopción en una amplia gama de tareas. Estas aplicaciones abarcan desde el procesamiento del lenguaje natural y la visión por computadora hasta la predicción de series temporales y la toma de decisiones, consolidando a las redes neuronales como una tecnología esencial en desarrollos tan diversos como los sistemas de recomendación y la conducción autónoma.

Aunque la literatura actual ha abordado ampliamente las redes neuronales convencionales, la creciente demanda de resolver problemas más complejos ha impulsado la evolución hacia arquitecturas más especializadas. Entre estas, las Redes Neuronales Gráficas (Graph Neural Networks o GNN) se han destacado por su capacidad para trabajar con datos estructurados en forma de gráficas, lo que las hace especialmente útiles para el objetivo de este trabajo.

Este capítulo ofrecerá una introducción formal a las redes neuronales gráficas y explorará la necesidad de adoptar estas estructuras avanzadas para el estudio de dominios donde la naturaleza de los datos es inherentemente no lineal o interdependiente.

#### 3.1. Redes Neuronales

A continuación se dará la definición formal del modelo de red neuronal partiendo de la unidad más básica denominada como perceptrón

**Definición 23** (Perceptrón). Sea  $f : \mathbb{R}^d \longrightarrow \{0, 1\}$  una función dada por:

$$f(x) = \begin{cases} 1 & \text{si } \mathbf{w}^T \mathbf{x} + b > 0 \\ 0 & \text{si } \mathbf{w}^T \mathbf{x} + b \leq 0 \end{cases}$$

Donde  $\mathbf{w}, \mathbf{x} \in \mathbb{R}^d$ :

- $\mathbf{x}$  representa el vector de entradas del perceptrón.
- $\mathbf{w}$  corresponde con el vector de pesos de cada entrada.
- $b$  es el umbral que se considera para una mejor flexibilidad a la hora de implementar el perceptrón.

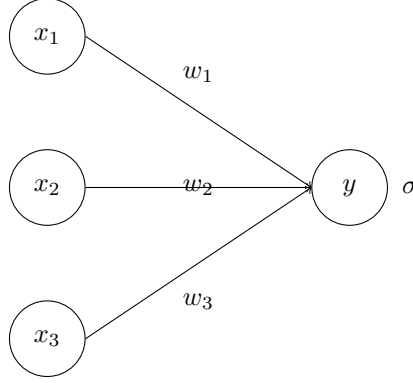


Figura 6: Ejemplo de perceptrón

**Definición 24** (Capa). Sea  $\mathbf{W} \in \mathbb{R}^{n \times m}$  una matriz de pesos,  $\mathbf{h} \in \mathbb{R}^n$  un vector de entradas (resultado de la salida de una capa anterior) y  $\mathbf{b} \in \mathbb{R}^n$  un vector de bias,  $\sigma : \mathbb{R} \rightarrow \mathbb{R}$  una función de activación. La capa  $L$ -ésima  $f^{(L)}$  de una red neuronal, está definida por:

$$f^{(L)} = \sigma \circ \mathbf{h}^{(L)}$$

Donde:

- $\mathbf{h}^{(L)} = \mathbf{W}^{(L)} f^{(L-1)} + b^{(i)}$  es la combinación lineal entre la matriz de pesos  $\mathbf{W}$  que conecta la capa  $L$  con la capa  $L-1$  y la salida de la capa anterior.
- $\sigma(\cdot)$  es la función de activación.
- $\mathbf{h}^{(0)} = \mathbf{X}$  tal que  $\mathbf{X} \in \mathbb{R}^d$ , con  $d$  el número de entradas.

**Definición 25** (Red Neuronal). Una **Red Neuronal** con  $\mathbf{L}$  capas es la composición de las funciones  $f_i$ ,  $i \in \{1, \dots, \mathbf{L}\}$  tales que:

$$y = f(x; \theta) = f^{(\mathbf{L})}(f^{(\mathbf{L}-1)}(\dots f^{(2)}(f^{(1)}(x))))$$

Donde:

- $\mathbf{x}$  es el vector de entrada.
- $f_i$  es la transformación realizada por la capa  $i$ -ésima.
- $\theta$  son los parámetros de la red (pesos y bias).
- $\mathbf{y}$  es la salida.

**Definición 26** (Función de pérdida). Sea  $y$  el valor verdadero del resultado de una tarea de aprendizaje,  $\hat{y} = f(x; \theta)$  una red neuronal, entonces la función de pérdida

$$\mathbf{L}(y, \hat{y})$$

mide la diferencia entre ambos, valor verdadero y valor predicho, donde el objetivo es encontrar los parámetros  $\theta$  que minimicen dicha función, es decir:

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i; \theta))$$

**Ejemplo 2.** La función de pérdida puede variar dependiendo de la tarea que se realice, aquí tenemos una serie de ejemplo de dicha función dependiendo del contexto

- **Error cuadrático medio (MSE)**

Esta función de pérdida es comúnmente usada en problemas de regresión. Mide la diferencia entre los valores  $y$ ,  $\hat{y}$  elevando al cuadrado dicha diferencia:

$$L(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Error absoluto medio (MAE)**

Utilizada también para tareas de regresión que mide la magnitud de los errores de manera más robusta, se define como:

$$L(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Entropía cruzada**

Esta función es utilizada habitualmente en problemas de clasificación midiendo la similitud entre la probabilidad predicha  $\hat{y}$  y la real  $y$ , de forma generalizada para  $\mathbf{K}$  número de clases se tiene que:

$$L(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

**Definición 27** (Función de riesgo). Sea  $\mathbf{X}$  el vector de características de entrada y sea  $\mathbf{y}$  la variable objetivo, dada la red neuronal  $f(x; \theta)$  la función de riesgo se define cómo:

$$\mathcal{R}(\theta) = \mathbb{E}[\mathbf{L}(y, f(x; \theta))]$$

Donde:

- $\mathbb{E}$  es la esperanza de la distribución conjunta  $(\mathbf{x}, \mathbf{y})$ .
- $\mathbf{L}(y, f(x; \theta))$  es la función de pérdida, entre  $y$ ,  $f(x; \theta)$

Dado que en la práctica no conocemos la verdadera distribución  $p(x, y)$  no podemos calcular directamente el riesgo esperado. En su lugar definimos otro tipo de función de riesgo:

**Definición 28** (Función de riesgo empírico). Sea  $\{(x_i, y_i)\}, i \in \{1, \dots, N\}$  un conjunto finito de datos de entrenamiento, la función de riesgo empírico está dada por:

$$\hat{\mathcal{R}}(\theta) = \frac{1}{N} \sum_{i=1}^N \mathbf{L}(y_i, f(x_i; \theta))$$

Una manera de minimizar las funciones de pérdida, es utilizando el algoritmo de Gradiente Descendente.

**Definición 29** (Gradiente). Sea  $f : \Omega \subset \mathbb{R}^n \rightarrow \mathbb{R}$ , una función diferenciable en  $x_0$ . Entonces el vector cuyas compnentes son las derivadas parciales de  $f$  en  $x_0$  es el vector gradiente, denotado por  $\nabla f$ , donde:

$$\nabla f(x_0) = \left( \frac{\partial f(x_0)}{\partial x_1}, \dots, \frac{\partial f(x_0)}{\partial x_n} \right)$$

Este vector contiene la información de cuanto crece o decrece la función en un punto en específico.

**Definición 30** (Descenso por gradiente). Sea  $J : \Theta \rightarrow \mathbb{R}$  una función objetivo, donde  $\Theta$  es el espacio de soluciones,  $\eta$  la tasa de aprendizaje del modelo que determina el tamaño de los pasos en cada iteración, entonces, se define al gradiente descendente cómo un método para encontrar el mínimo de una función de forma iterativa, se basa en iterar los valores de  $\theta$  en la dirección negativa del gradiente de  $J(\theta)$  de la siguiente forma:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$$

A continuación definiremos el algortimo que se utiliza para la actualización de los valores  $\theta$  utilizando el gradiente descendente: Existen otras variantes del Descenso de Gradiente,

---

**Algorithm 1** Algoritmo de actualización GD

---

**Inputs:**

$J : \Theta \rightarrow \mathbb{R}$ ,  $\mathbf{T} > 0$  número de iteraciones,  $\eta$  rango de aprendizaje.

**Initialize:**

Seleccionar  $\theta^{(0)} \in \Theta$  aleatoriamente.

**for**  $t \in \{1, \dots, \mathbf{T}\}$  **do**

Calcular  $R(\theta^{(t)})$ ,  $\nabla_{\theta^{(t)}} R(\theta^{(t)})$

**if**  $\nabla_{\theta^{(t)}} R(\theta^{(t)}) \neq 0$  **then**

$\theta^{(t+1)} \leftarrow \theta^{(t)} - \eta \nabla_{\theta^{(t)}} R(\theta^{(t)})$

**else**

Fin del algoritmo

**end if**

**end for**

---

pero para efectos de esta tesis, nos quedaremos con la definición más básica. A medida que va aumentando el número de capas en la red neuronal, también aumenta la complejidad de determinar los pesos de todas las conexiones de manera correcta para minimizar los errores, es por eso que nace la necesidad de implementar algoritmos más complejos para la optimización de redes multicapa, de ahí que nace el **BackPropagation**. (Pendiente)

**Definición 31** (Derivada función de riesgo). Sea  $f(x; \theta)$  una red neuronal con parametros  $\theta$ , la derivada de su función de riesgo  $R(\theta)$  se define de la siguiente forma:

$$\nabla_{\theta} R(\theta) = \nabla f^{(\mathbf{L})} \cdot \nabla f^{(\mathbf{L}-1)} \dots \nabla f^{(2)} \cdot \nabla f^{(1)}$$

Notamos que esta definición es posible gracias a la regla de la cadena.

(anexo)

**Observación 1** (Propagación hacia adelante). Ya sabemos que para cada capa ( $L$ ) de una red neuronal, la salida  $\mathbf{h}^{(L)}$  se calcula de manera recursiva, dado un vector de entrada  $\mathbf{x}$  la red realiza las siguientes dos operaciones:



■ **Preactivación**

Dada la matriz de pesos  $\mathbf{W}^{(L)}$ , el vector de sesgos  $\mathbf{b}^{(L)}$  en la capa  $L$  y  $\mathbf{h}^{(L-1)}$  la salida de la capa anterior, entonces la preactivación está dada por:

$$\mathbf{z}^{(L)} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

■ **Activación**

Dada  $\sigma$  una función de activación (sigmoide, ReLu, etc), y  $\mathbf{z}^{(L)}$  la preactivación de la capa  $L$ , entonces la activación en la capa  $L$  está dada por:

$$\mathbf{h}^{(L)} = \sigma(\mathbf{z}^{(L)})$$

De esta forma la salida de la última capa se compara con la etiqueta verdadera y utilizando alguna función de pérdida  $\mathbf{J}(\mathbf{h}^{(L)}, y)$

**Observación 2** (Minimización del error en la capa de salida). Para minimizar el error en la función de pérdida utilizamos la definición 32, por una serie de pasos dados de la siguiente manera:

■ **Error en la capa de salida**

Sea  $\frac{\partial \mathbf{J}}{\partial \mathbf{h}^{(L)}}$  el gradiente de la función de pérdida con respecto a la salida de la red,  $\sigma'(\mathbf{z}^{(L)})$ , la derivada de la función de activación de la última capa, entonces el error en la capa de salida está dado por:

$$\delta^{(L)} = \frac{\partial \mathbf{J}}{\partial \mathbf{z}^{(L)}} = \frac{\partial \mathbf{J}}{\partial \mathbf{h}^{(L)}} \cdot \sigma'(\mathbf{z}^{(L)})$$

■ **Propagación del error hacia atrás**

Sea  $\delta^{(L+1)}$  el error de la capa siguiente y  $\sigma'(\mathbf{z}^{(L)})$  la derivada de la función de activación de la capa  $L$ , entonces la propagación del error se da mediante la siguiente expresión:

$$\delta^{(L)} = \left( \mathbf{W}^{(L+1)} \right)^T \delta^{(L+1)} \cdot \sigma'(\mathbf{z}^{(L)})$$

■ **Gradiente de los parámetros**

Para calcular los gradientes de los parametros, vemos que se realiza de la siguiente forma:

- Sea  $\mathbf{h}^{(L-1)}$  la salida de la capa anterior, entonces el gradiente de los pesos está dado por:

$$\frac{\partial \mathbf{J}}{\partial \mathbf{W}^{(L)}} = \delta^{(L)} \cdot \left( \mathbf{a}^{(L-1)} \right)^T$$

- Y para los sesgos se calcula de la siguiente forma:

$$\frac{\partial \mathbf{J}}{\partial \mathbf{b}^{(L)}} = \delta^{(L)}$$

■ **Actualización de los parámetros**

Una vez que se tienen los pesos y sesgos calculados de la red, se actualizan utilizando un método de optimización ya mencionado anteriormente llamado descenso del gradiente, de esta forma la actualización está dada por:

- *Pesos:*

$$W^{(L)} = W^{(L)} - \eta \frac{\partial \mathbf{J}}{\partial W^{(L)}}$$

- *Bias:*

$$b^{(L)} = b^{(L)} - \eta \frac{\partial \mathbf{J}}{\partial b^{(L)}}$$

Con lo observado anteriormente, podemos definir el algoritmo **backpropagation** enunciado en el siguiente pseudocódigo:

---

**Algorithm 2** Algoritmo Backpropagation

---

```

1: Initialize Inicializamos pesos  $W^{[l]}$  y bias  $b^{[l]}$  de manera aleatoria para cada capa  $l = 1, \dots, L$ 
2: for cada época do
3:   for cada ejemplo de entrenamiento  $(x, y)$  do
4:     Propagación hacia adelante:
5:     Asignamos  $a^{[0]} = x$ 
6:     for each layer  $l = 1, \dots, L$  do
7:        $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$ 
8:        $a^{[l]} = g(z^{[l]})$ 
9:     end for
10:    Calculamos la pérdida  $J(a^{[L]}, y)$ 
11:    Backward Pass:
12:    Calculamos el error para cada capa de salida:
13:     $\delta^{[L]} = \frac{\partial J}{\partial a^{[L]}} \cdot g'(z^{[L]})$ 
14:    for cada capa  $l = L - 1, \dots, 1$  do
15:      Propagación del error hacia atrás:
16:       $\delta^{[l]} = (W^{[l+1]})^\top \delta^{[l+1]} \cdot g'(z^{[l]})$ 
17:      Calculamos los gradientes:
18:       $\frac{\partial J}{\partial W^{[l]}} = \delta^{[l]} \cdot (a^{[l-1]})^\top$ 
19:       $\frac{\partial J}{\partial b^{[l]}} = \delta^{[l]}$ 
20:    end for
21:    Actualización de parámetros (Descenso del gradiente):
22:    for each layer  $l = 1, \dots, L$  do
23:       $W^{[l]} = W^{[l]} - \eta \cdot \frac{\partial J}{\partial W^{[l]}}$ 
24:       $b^{[l]} = b^{[l]} - \eta \cdot \frac{\partial J}{\partial b^{[l]}}$ 
25:    end for
26:  end for
27: end for

```

---

Así, hemos definido los componentes de una red neuronal, su forma de optimización y su entrenamiento.

### 3.2. Redes convolucionales CNN

Las **redes neuronales convolucionales (CNN)** son una clase especializada de redes neuronales diseñadas para procesar datos que poseen una estructura de malla o grilla, como imágenes y series de tiempo. Estas redes se emplean comúnmente en tareas de procesamiento de imágenes, donde los datos se organizan en una malla bidimensional, o en el caso de las series temporales, que pueden representarse como una malla unidimensional con muestras tomadas a lo largo de intervalos de tiempo.

A diferencia de las redes neuronales tradicionales, donde se utiliza la multiplicación general de matrices, las CNN aplican la operación de convolución en al menos una de sus capas. Esto les permite extraer automáticamente características locales de los datos, como bordes, texturas o patrones, lo que mejora significativamente su capacidad para procesar datos con una estructura espacial clara.

Estudiar las CNN es fundamental como antecedente para entender las redes gráficas convolucionales (GCN), ya que comparten la idea central de la convolución, pero aplicadas a estructuras de datos más complejas, como los grafos. En esta sección, exploraremos la importancia de las CNN como base teórica, profundizando en su arquitectura y en las matemáticas que subyacen a este tipo particular de redes. La mayoría de las definiciones fueron obtenidas del libro [3].

**Definición 32** (Convolución). *Sean  $x, w : \mathbb{R} \rightarrow \mathbb{R}$  dos funciones. La convolución de  $x$  y  $w$  denotada por:  $(x * w)(t)$  es una operación binaria que produce una nueva función dada por:*

$$s(t) = (x * w)(t) = \int x(a)w(t - a)da$$

En el contexto de las redes neuronales convolucionales, el primer término (la función  $x$ ) se refiere a la entrada, y el segundo término (la función  $w$ ) es el kernel o filtro. Dado que en muchos escenarios, el tipo de datos es discreto, se maneja otro tipo de convolución, llamada convolución discreta.

**Definición 33** (Convolución discreta). *Sean  $x(n), w(n)$  dos sucesiones, la convolución discreta, se define como:*

$$(x * w)(n) = \sum_{a=-\infty}^{\infty} x(n)w(n - a)$$

Finalmente, para el uso particular del procesamiento de imágenes en dos dimensiones, podemos utilizar la convolución discreta en dos dimensiones.

**Definición 34** (Convolución en dos dimensiones). *Sean  $\mathbf{I}$  y  $\mathbf{K}$  dos matrices definidas en  $\mathbf{M}_{22}$ , la convolución de matrices discretas se define cómo:*

$$\mathbf{S}(i, j) = (\mathbf{I} * \mathbf{K})(i, j) = \sum_m \sum_n \mathbf{I}(m, n) \mathbf{K}(i - m, j - n)$$

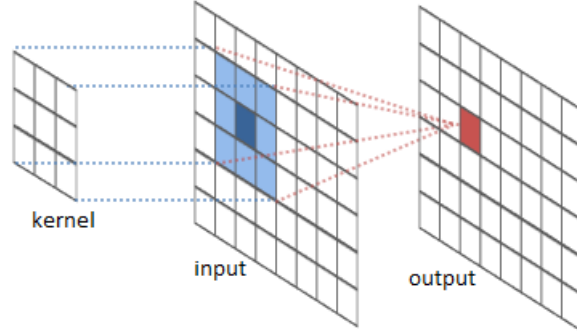
**Observación 3.** *Notamos que, como la convolución es conmutativa, entonces la definición anterior puede reescribirse de tal forma que:*

$$\mathbf{S}(i, j) = (\mathbf{K} * \mathbf{I})(i, j) = \sum_m \sum_n \mathbf{I}(i - m, j - n) \mathbf{K}(m, n)$$

**Definición 35** (Kernel (Filtro)). Sea  $\mathbf{X} \in \mathbb{R}^{m \times n}$  un conjunto de datos de entrada. El kernel (o filtro)  $\mathbf{K}$  es una matriz de pesos de tamaño  $h \times w$  donde  $m \geq h, n \geq w$ . Tal que actúa sobre la entrada  $\mathbf{X}$  mediante una convolución, es decir:

$$(\mathbf{K} * \mathbf{X})(i, j) = \sum_{u=1}^h \sum_{v=1}^w \mathbf{X}(i + u - 1, j + v - 1) \mathbf{K}(u, v)$$

Donde se introduce el término  $-1$  para alinear el kernel con la entrada de manera que este quede centrado.



A continuación, enunciaremos algunas propiedades clave de los filtros.

**Definición 36** (Propiedades del kernel). Sea  $\mathbf{K}$  un filtro de dimensiones  $h \times w$ .

- **Dimensión** La dimensión del kernel está dado por:

$$\dim(\mathbf{K}) = h \times w$$

- **Simetría** Un kernel es simétrico si satisface:

$$\mathbf{K}(i, j) = \mathbf{K}(h - i, w - j) \quad \forall i, j \in \{0, \dots, h\}, \{0, \dots, w\}$$

- **Normalización** Se dice que un kernel está normalizado si :

$$\sum_{i=0}^h \sum_{j=0}^w \mathbf{K}(i, j) = 1$$

**Definición 37** (Mapa de características). Sea  $\mathbf{I}$  un tensor de entrada,  $\mathbf{K}$  el kernel, el mapa de características  $\mathbf{F}$  está dado por:

$$\mathbf{F}(i, j) = (\mathbf{I} * \mathbf{K})(i, j)$$

**Definición 38** (Stride). Sea  $\mathbf{K}$  un filtro de tamaño  $h \times w$  aplicado en una entrada  $\mathbf{I}$ , entonces el stride  $\mathbf{s}$  para cada entrada  $i, j$  está determinado por:

$$(i \cdot s, j \cdot s)$$

por lo cual el mapa de características quedaría determinado por :

$$\mathbf{F}(l, m) = \mathbf{F}(i \cdot s, j \cdot s) \quad l = i \cdot s, m = j \cdot s$$

**Definición 39** (Padding). Sea  $\mathbf{I}$  una entrada de dimensiones  $m \times n$  el padding  $\mathbf{p}$  de tamaño  $\rho$  es una función  $\mathbf{p}_\rho : \mathbb{R}^{m \times n} \longrightarrow \mathbb{R}^{(m+2\rho) \times (n+2\rho)}$  tal que:

$$\mathbf{p}_\rho(\mathbf{I})(i, j) = \begin{cases} \alpha, & \text{si } i \leq \rho, j \leq \rho, i > m + \rho, j > n + \rho \\ \mathbf{I}(i - \rho, j - \rho), & \text{en otro caso.} \end{cases}$$

Donde  $\alpha \in \mathbb{R}$ .

**Definición 40** (Invarianza). Sea  $f : \mathbf{X} \longrightarrow \mathbf{Y}$  una función,  $\mathbf{T}$  una transformación en  $\mathbf{X}$ . Se dice que una función  $f$  es invariante bajo  $\mathbf{T}$  si:

$$f(\mathbf{T}(x)) = f(x), \quad \forall x \in \mathbf{X}$$

**Definición 41** (Equivarianza). Sean  $f, g$  dos funciones tales que  $f : \mathbf{X} \longrightarrow \mathbf{Y}, g : \mathbf{Y} \longrightarrow \mathbf{X}$ . Se dice que  $f$  es equivariante bajo  $g$  si:

$$f(g(x)) = g(f(x)) \quad \forall x \in \mathbf{X}$$

De las definiciones anteriores, podemos notar que están muy relacionadas ya que al aplicar convoluciones a datos estructurados podemos notar que tiene la propiedad de ser equivariante con respecto a ciertas transformaciones, por ejemplo a la traslación.

**Observación 4.** Sea  $f$  una función y consideremos la transformación  $\mathbf{T}_a$  cómo la traslación de su entrada en la cantidad  $a$ , es decir:  $\mathbf{T}_a f(x) = f(x - a)$ . Aplicando la convolución tenemos que:

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} \mathbf{T}_a f(t) k(x - t) dt$$

Sustituyendo el valor de la transformación obtenemos:

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(t - a) k(x - t) dt$$

Y realizando el cambio de variable  $u = t - a$  tenemos que  $du = dt$  y además  $t = u + a$  así:

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k(x - (u + a)) du$$

Que, reordenando los términos obtenemos lo siguiente:

$$(\mathbf{T}_a f * k)(x) = \int_{\mathbb{R}} f(u) k((x - a) - u) du$$

Que por definición tenemos que:

$$(f * k)(x - a) = \int_{\mathbb{R}} f(u) k((x - a) - u) du$$

Por lo tanto:

$$(\mathbf{T}_a f * k)(x) = (f * k)(x - a) \quad \forall x \in \mathbb{R}$$

De esta forma, demostramos que la operación convolución para funciones definidas en  $\mathbb{R}$  son equivariantes con respecto a las traslaciones. Es decir, que en el contexto de las redes neuronales, si trasladamos la entrada de la red, significa que la salida estará trasladada en la misma medida, lo cual es útil cuando se aplica a múltiples ubicaciones en la entrada.

**Definición 42** (Max Pooling). Sea  $\mathbf{X}$  una entrada en  $\mathbb{R}^{m \times n}$ ,  $\mathbf{s}$  un stride dado,  $\mathbf{M} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{p \times q}$  una función, entonces la función  $\mathbf{M}$  está dada por:

$$\mathbf{M}(\mathbf{X})(i, j) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{X}(i \cdot s + u, j \cdot s + v)$$

**Definición 43** (Average pooling). Sea  $\mathbf{X}$  una entrada en  $\mathbb{R}^{m \times n}$ ,  $\mathbf{s}$  un stride dado,  $\mathbf{A} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{p \times q}$  una función, entonces la función  $\mathbf{A}$  está dada por:

$$\mathbf{A}(\mathbf{X})(i, j) = \frac{1}{h \cdot w} \sum_{u=0}^{h-1} \sum_{v=0}^{w-1} \mathbf{X}(i \cdot s + u, j \cdot s + v)$$

Hasta ahora, se ha visto como las operaciones de convolución y pooling extraen las características de una red neuronal, una vez realizadas estas operaciones, el resultado se aplana en un único vector con el objetivo de utilizado como entrada para una capa completamente conectada donde se realizará la tarea que se requiera.

**Definición 44** (Arquitectura de una CNN). Sea  $f : \mathbb{R}^{m \times n \times d} \rightarrow \mathbb{R}^k$  una función, la arquitectura de una red neuronal convolucional (CNN) se define como la composición de tres funciones donde la composición está dada por:

$$\mathbf{y} = \mathbf{g} \circ \mathbf{P} \circ \mathbf{C}(\mathbf{X})$$

donde:

- $\mathbf{C}(\mathbf{X}) = (\mathbf{K} * \mathbf{X}) + \mathbf{b}$  es la capa convolucional.
- $\mathbf{P}(\mathbf{C}(\mathbf{X})) = \max_{0 \leq u < h, 0 \leq v < w} \mathbf{C}(\mathbf{X})(i \cdot s + u, j \cdot s + v)$  Representa la capa de pooling y para este caso, como ejemplo se utiliza el máx pooling.
- $\mathbf{y} = \mathbf{g}(\mathbf{WP} + \mathbf{b})$  es la capa completamente conectada.

### 3.3. Redes Neuronales Gráficas

**Definición 45** (Encoder). Sea  $G$  una gráfica, un encoder es una función  $\phi$  tal que  $\phi : \mathcal{V} \rightarrow \mathbb{R}^d$  dada por:

$$\phi(v, \mathcal{N}(v), G(E)) = z_v \quad \text{donde } z_v \in \mathbb{R}^d$$

Por simplicidad denotaremos  $\phi(v, \mathcal{N}(v), E(v)) := \phi(v)$

**Definición 46** (Matriz de encajes). Sea  $v \in \mathcal{V}$  tenemos que:

$$\phi(v) = \mathbf{Z}[v] \quad \text{donde } \mathbf{Z} \in \mathbb{R}^{v_G \times d}$$

es una matriz que contiene los vectores de encajes  $z_v$  correspondientes a cada nodo de la gráfica.

**Definición 47** (Decoder). Sea  $z_v \in \mathbb{R}^d$  un encaje producido por un encoder, entonces la función  $\psi : \mathbb{R}^d \rightarrow Y$  se denomina decoder, y produce una predicción en el espacio de salida  $Y$ , es decir:

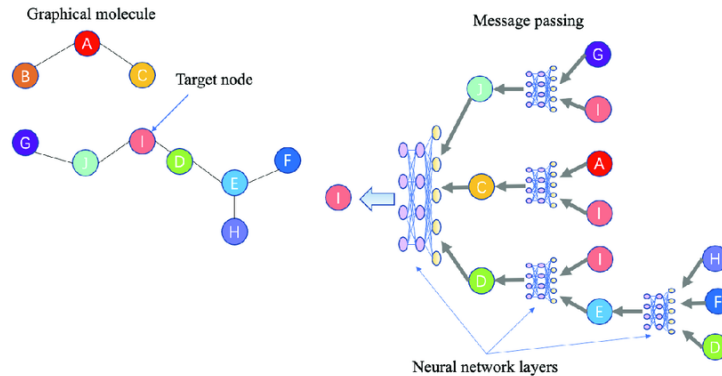
$$\psi(z_v) = \hat{y}$$

### 3.4. Propagación de Mensajes Neuronales

El concepto de **Propagación de Mensajes Neuronales** se refiere al mecanismo mediante el cual las representaciones de los vértices de una gráfica, se actualizan mediante el intercambio de información de su vecindario. En cada paso del mecanismo, los nodos intercambian mensajes que contienen información sobre el estado en que se encuentran para luego combinarse y actualizar su información interna.

Esto viene motivado por la necesidad de capturar relaciones y dependencias entre los vértices de una gráfica para estudiar su comportamiento estructural.

En diferentes contextos del mundo actual, notamos que muchos de los datos tienen una estructura propia de gráfica, tales como: redes sociales, moléculas, sistemas de recomendación, etc, siendo los vértices entidades como usuarios, átomos o productos de un e-commerce y las aristas las relaciones entre ellos tales como, amistades, enlaces y preferencias. El concepto de **Propagación de Mensajes Neuronales**, permite estudiar a la gráfica, no sólo a nivel de característica de vértice, sino también a nivel estructural de gráfica.



### 3.5. Descripción general del mecanismo

Cómo se mencionó en la motivación, durante cada iteración del mecanismo en una GNN (Red Neuronal Gráfica), los encajes ocultos  $h_u^{(k)}$  correspondientes a cada nodo  $u \in V$ , son actualizadas de acuerdo a la información recopilada del vecindario de  $u$ .

El mecanismo puede ser definido de la siguiente forma:

**Definición 48** (Propagación de mensajes neuronales). Sea ,  $G = (V, E)$  una gráfica,  $N_G(v)$  el vecindario del nodo  $v \in V$  y  $h_u^{(t)}$  los encajes ocultos del nodo  $v$ , y  $\mathcal{A}$  una función de agregación que combina los encajes de los vecinos de  $v$  entonces, el mecanismo de propagación de mensajes está dado por:

$$m_v^{(t)} = \mathcal{A}(h_u^{(k)})$$

Una vez que el vértice a recibido el mensaje, la actualización de su información es el resultado de la combinación del mensaje recibido, con la información actual que tenía y se define de la siguiente forma:

**Definición 49.** Sea  $h_v^{(t-1)}$  el encaje oculto de la capa anterior  $(t-1)$ , y  $m_v^{(t)}$  la propagación del mensaje en la capa  $t$ , entonces la actualización del mensaje está dado por:

$$h_v^{(t)} = \mathcal{U}(h_v^{(t-1)}, m_v^{(t)})$$

Donde  $\mathcal{U}$  es una función de actualización, normalmente es una red neuronal, con una capa lineal seguida de una función de activación (ReLU)

Después de  $K$  iteraciones del mecanismo, podemos utilizar la salida del final de la capa para definir los encajes de cada vértice, es decir:

$$z_u = h_u^{(K)}.$$

### 3.6. La GNN fundamental

Hasta el momento, se ha estudiado el marco de las GNN de una manera un poco abstracta utilizando las funciones que definimos en la sección anterior e iterando  $K$  veces.

Para introducir una manera de abordarlas de forma más general, introducimos el modelo GNN más básico. El modelo GNN fundamental es definido por:

$$h_u = \phi \left( x_u, \bigoplus_{v \in \mathcal{N}(u)} \psi(x_u, x_v) \right)$$

Donde:

- $h_u$  es la nueva representación del nodo  $u$  después de aplicar una capa de la GNN
- $x_u$  Representa la característica del nodo  $u$  antes de la actualización.
- $\psi(x_u, x_v)$  Es una función de transformación que toma las características del nodo  $u$  y de su vecino  $v$ , produciendo una especie de mensaje o interacción entre ambos nodos.
- $\bigoplus_{v \in \mathcal{N}(u)}$  Es una operación de agregación como puede ser suma, media, max pooling, etc, sobre los vecinos de  $u$  es decir sobre las transformaciones producidas por la función  $\psi(\cdot)$ .



- $\phi(\cdot)$  es la función de actualización que toma las características originales del nodo  $u(x_u)$  y el resultado de la agregación de los mensajes de sus vecinos para producir una nueva representación del nodo  $u$ .

En términos generales, el comportamiento lo podemos describir a continuación:

1. **Mensajes entre nodos vecinos** ( $\psi(x_u, x_v)$ )

La función  $\psi$  calcula la interacción entre las características del nodo  $u$  y las características de cada uno de sus vecinos  $v$ . Esto permite capturar la relación entre los nodos de forma que dependa de sus características.

2. **Agregación** ( $\oplus$ )

Después de calcular las interacciones entre  $u$  y sus vecinos, estas interacciones se agregan mediante una operación  $\oplus$ , que comúnmente (cómo se vió anteriormente) es una suma, media o máximo. La agregación tiene como objetivo combinar toda la información recibida entre los vecinos de  $u$  en un solo término.

3. **Actualización** ( $\phi$ )

Finalmente, la función  $\phi$  toma la característica original del nodo  $u, x_u$  y el resultado de la agregación para producir la nueva representación  $h_u$ . La función  $\phi$  puede ser una red neuronal feedforward o cualquier otra transformación no lineal que aprenda a integrar la información de  $u$  con la de sus vecinos.

En resumen, el modelo anterior expresa el proceso de **message passing** de una GNN, donde los mensajes de los vecinos de un nodo se combinan para actualizar la representación del nodo central, aprovechando las simetrías presentes en las gráficas.

### 3.7. Redes Gráficas Convolucionales (GCNs)

**Definición 50.** *Arquitectura de una GNN] Sea  $G$  una gráfica,  $A$  su matriz de adyacencia  $F$  una función de activación,  $\Phi$  los parámetros de la red, entonces la arquitectura de una GCN está definida por:*

$$\begin{aligned} \mathbf{H}_1 &= \mathbf{F}[\mathbf{X}, \mathbf{A}, \Phi_0] \\ \mathbf{H}_2 &= \mathbf{F}[\mathbf{H}_1, \mathbf{A}, \Phi_1] \\ &\vdots \\ \mathbf{H}_K &= \mathbf{F}[\mathbf{H}_{K-1}, \mathbf{A}, \Phi_{K-1}] \end{aligned}$$

Donde  $\mathbf{X}$  es la entrada y  $\mathbf{H}_K$  contiene los encajes de nodos modificados.

**Definición 51** (Equivarianza). *Sea  $F$  una función de activación  $H$  la matriz de encajes ocultos de nodos,  $P$  una matriz de permutación, decimos que una capa es equivariante si:*

$$\mathbf{H}_{K+1} \mathbf{P} = \mathbf{F}[\mathbf{H}_K \mathbf{P}, \mathbf{P}^T \mathbf{A} \mathbf{P}, \Phi_K]$$

Donde  $A$  es la matriz de adyacencia para la gráfica  $G$

## 4. Muestreo General

En los últimos años, los avances en el campo del *Deep Learning* y la Inteligencia Artificial han transformado distintas disciplinas como la visión por computadora, el análisis de datos y el reconocimiento de patrones. Estas herramientas se han construido con bases matemáticas sólidas, como el Análisis matemático, el álgebra lineal y la estadística, las cuales han sido ampliamente documentadas y desarrolladas en textos con el que se ha citado en repetidas ocasiones en esta Tesis *Deep Learning* de Goodfellow et al (2016). Sin embargo la relación entre las matemáticas y el aprendizaje profundo no se limita únicamente al uso de conceptos matemáticos para el desarrollo de modelos; por el contrario, podemos utilizar dichos modelos para explorar y analizar estructuras de objetos matemáticos complejos (Bronstein et al, 2021). Este capítulo aborda el estudio de nudos de Lissajous, un tipo especial de nudo generado por funciones armónicas tridimensionales que se modelan mediante parámetros periódicos, los cuales presentan una serie de configuraciones topológicas que pueden ser modeladas y analizadas a través de herramientas modernas de aprendizaje automático y teoría de gráficas. En este contexto, el objetivo principal es clasificar con base en una serie de propiedades de estos nudos, de las cuales dentro de todas estas se encuentra la característica de Euler, una propiedad topológica fundamental que describe la conectividad y la estructura de estos objetos. Para ello, emplearemos arquitecturas avanzadas de aprendizaje profundo, como las Redes Neuronales Gráficas que hemos descrito en capítulos anteriores, que han demostrado ser herramientas efectivas para analizar estructuras no euclidianas (Xu et al (2019)). A lo largo de este capítulo, se detallará la metodología empleada, que combina técnicas de visualización de gráficas, empleo de algoritmos de clasificación utilizando GNNs, así como una serie de algoritmos de muestreo que se utilizan para obtener mejores resultados. Este enfoque resalta cómo las herramientas existentes de Deep Learning pueden extenderse más allá de los problemas tradicionales de clasificación y predicción, ofreciendo nuevas perspectivas de resolver problemas fundamentales en matemáticas y teoría de nudos.

### 4.1. Superficies estudiadas

La metodología seguida para estudiar nudos de Lissajous desde un enfoque basado en gráficas comienza con la generación de superficies en  $\mathbb{R}^3$  a partir de las curvas de Lissajous "engordadas" para convertirlas en objetos con volumen. Estos objetos se triangulan utilizando la librería **trimesh** del lenguaje de programación Python, una herramienta robusta para manipulación y análisis de geometrías tridimensionales. Este paso permite representar la superficie del nudo mediante una malla compuesta por vértices y caras triangulares. Posteriormente esta malla se convierte en una gráfica mediante otra librería de Python llamada **networkX**, donde cada nodo representa un vértice de la superficie triangulada y cada arista define la conectividad entre vértices adyacentes en la superficie. Esta representación gráfica es fundamental para analizar las propiedades estructurales de los nudos ya que facilita su visualización y procesamiento mediante técnicas de aprendizaje profundo basadas en gráficas, como las GNNs. A lo largo del proceso, a parte de utilizar la librería **networkX** para la construcción, se utiliza **matplotlib** para su visualización, lo que permite inspeccionar visualmente los resultados y evaluar las características topológicas de los nudos modelados.

## 4.2. Métodos de Muestreo

Durante el análisis de las estructuras de la grafica, se identificó un problema relacionado con el procesamiento debido a la naturaleza del entorno local. Para abordar este desafío, se diseñaron una serie de métodos y técnicas que permitieron realizar un muestreo efectivo sobre las superficies y nodos del grafo, sin comprometer la preservación de su estructura original.

## 4.3. Orden por árbol generador mínimo

Uno de los desafíos al obtener muestras de la estructura de un grafo es definir un método de recorrido que evite imponer un orden arbitrario y garantice que los nodos no sean visitados más de una vez. Esto es esencial para prevenir sesgos en los resultados derivados del orden en que se procesan los nodos.

Con el objetivo de realizar un muestreo aleatorio sin un orden predeterminado, se han desarrollado diversas técnicas y métodos. Estas técnicas surgen de la combinación de dos conceptos previamente introducidos y detallados en el capítulo 2. En primer lugar, generamos un árbol generador mínimo con base en la matriz adyacencia  $A$  de la gráfica  $G$ , por medio del algoritmo de Kruskal, descrito a continuación:

---

**Algorithm 3** Algoritmo de Kruskal para el Árbol de Expansión Mínima (MST)

---

**Require:** Grafo no dirigido  $G = (V, E)$  con pesos en las aristas

**Ensure:** Árbol de expansión mínima (MST)

```
1: Inicializar  $MST \leftarrow \emptyset$  ▷ Conjunto vacío para el MST
2: Ordenar las aristas  $E$  en orden creciente de pesos
3: Inicializar estructura Union-Find para los vértices  $V$ 
4: for cada arista  $(u, v) \in E$  en orden creciente de peso do
5:   if  $\text{FIND}(u) \neq \text{FIND}(v)$  then ▷ Si  $u$  y  $v$  no están en el mismo componente
6:      $MST \leftarrow MST \cup \{(u, v)\}$  ▷ Añadir arista al MST
7:      $\text{UNION}(u, v)$  ▷ Unir los componentes de  $u$  y  $v$ 
8:   end if
9:   if Número de aristas en MST  $= |V| - 1$  then ▷ MST completo
10:    break
11:   end if
12: end for
13: return  $MST$  ▷ Devolver el árbol de expansión mínima
```

---

Posteriormente a utilizar el algoritmo de Kruskal para generar el árbol generador mínimo, convertimos el árbol resultante a una matriz simétrica de la siguiente forma :  $A_s = A_{mst} + A_{mst}^T$ . El objetivo de generar el árbol generador mínimo es recorrer la estructura de la gráfica completa sin recorrer todas las aristas, sino sólo las más importantes que en este caso, son las que dan la estructura a la gráfica original. Una vez obtenido el MST, procedemos a recorrerlo mediante un orden impuesto por el algoritmo "DFS", el cual es un algoritmo que sirve para recorrer una gráfica en profundidad, es decir a lo "largo" asegurando así que visitaremos todos los nodos de dicha gráfica para preservar lo que nos interesa que es la estructura de la gráfica. A continuación enunciaremos el algoritmo DFS, y su implementación en nuestro

método:

---

**Algorithm 4** DFS Order

---

**Require:** Nodo actual  $nodo$ , lista de nodos visitados  $visitados$ , lista del orden  $orden$

**Ensure:** Lista  $orden$  con el orden en que los nodos son visitados

```
1: function DFS_ORDER( $nodo, visitados, orden$ )
2:    $visitados[nodo] \leftarrow \text{True}$  ▷ Marcar el nodo como visitado
3:   Agregar  $nodo$  a la lista  $orden$  ▷ Registrar el nodo en el orden de visita
4:   for all  $vecino$  en posiciones donde  $mst[nodo] = 1$  do
5:     if  $visitados[vecino] = \text{False}$  then
6:       DFS_ORDER( $vecino, visitados, orden$ ) ▷ Llamada recursiva para el vecino
       no visitado
7:     end if
8:   end for
9: end function
```

---

Posterior a establecer el orden en el que se recorrerá la gráfica, generamos un orden entre nodos para visitar los nodos más lejanos del nodo actual, asegurando de esta forma ir lo más lejos posible en el menor número de pasos.

#### 4.4. Muestreo por Esferas

Una de las formas de mantener la estructura original de la gráfica es enfocándonos en su estructura local, es decir, podemos mirar a partir de un nodo  $v$  su vecindario, así como también tomar una pequeña esfera  $S_v(r)$  con radio  $r$  y centro en  $v$  que interseque su vecindario  $N_G(v)$ , para así mantener las propiedades locales y poder asegurar que se preserve la estructura recibida.

Para identificar la muestra resultado de la intersección entre  $N_G(V)$  y la esfera  $S_v(r)$  se ha construido un método denominado Método polar para desviaciones normales [5, p. 122-123]. A continuación describiremos un algoritmo que sirve como base a la implementación realizada para muestrear uniformemente puntos en el volumen de una esfera, se conoce cómo *Método polar para desviaciones normales* o *Método polar de Marsaglia*.

---

**Algorithm 5** Método polar para desviaciones normales

---

- 1: **P1** [Obtener variables uniformes]
- 2: Generar dos variables aleatorias independientes  $U_1, U_2 \sim U(0, 1)$ .
- 3: Asignar  $V_1 \leftarrow 2U_1 - 1$ ,  $V_2 \leftarrow 2U_2 - 1$ , ahora  $V_1, V_2 \sim U(-1, 1)$ .
- 4: **P2** [Calcular  $S$ ]
- 5: Asignar  $S \leftarrow V_1^2 + V_2^2$ .
- 6: **P3** [¿Es  $S \geq 1$ ?]
- 7: **if**  $S \geq 1$  **then**
- 8:     Regresar al paso **P1**.
- 9: **end if**
- 10: **P4** [Calcular  $X_1, X_2$ ]
- 11: Asignar:

$$X_1 \leftarrow V_1 \sqrt{\frac{-2 \ln S}{S}}, \quad X_2 \leftarrow V_2 \sqrt{\frac{-2 \ln S}{S}}$$

---

Para probar la validez del método anterior utilizaremos geometría analítica y un poco de cálculo.

*Demostración.* Notamos que si  $S < 1$  en el paso *P3*, el punto en el plano cartesiano con las coordenadas  $(V_1, V_2)$  es un punto aleatorio uniformemente distribuido dentro del círculo unitario.

Transformando a coordenadas polares tenemos que:  $V_1 = R \cos \theta$ ,  $V_2 = R \sin \theta$  y sustituyendo tenemos que:

$$S = R^2, \quad X_1 = (R \cos \theta) \sqrt{\frac{-2 \ln S}{R^2}}, \quad X_2 = (R \sin \theta) \sqrt{\frac{-2 \ln S}{R^2}}$$

De aquí observamos que:

$$\begin{aligned} X_1 &= (R \cos \theta) \sqrt{\frac{-2 \ln S}{R^2}} \\ &= R \cos \theta \frac{\sqrt{-2 \ln S}}{\sqrt{R^2}} \\ &= R \cos \theta \frac{\sqrt{-2 \ln S}}{|R|} \\ &= \cos \theta \sqrt{-2 \ln S} \end{aligned}$$

Pues  $R > 0$  De igual forma  $X_2 = \sin \theta \sqrt{-2 \ln S}$ . Utilizando coordenadas polares y haciendo:

$$R' = \sqrt{-2 \ln S}, \quad \theta' = \theta$$

Tenemos que  $X_1 = R' \cos \theta'$ ,  $X_2 = R' \sin \theta'$ . Así, observamos que  $R'$  y  $\theta'$  son independientes pues  $R$  y  $\theta$  lo son. También  $\theta' \sim U(0, 2\pi)$  pues es el rango de las funciones  $\sin$  y  $\cos$ .

A su vez notamos que:

$$\begin{aligned} P(R' \leq r) &= P(-2\ln S \leq r^2) \\ &= P(S \geq e^{-\frac{r^2}{2}}) \end{aligned}$$

Por otro lado, como  $S \sim U(0, 1)$  entonces su función de distribución acumulada es :

$$F(s) = P(S \leq s) = s, \quad s \in [0, 1]$$

De esta forma la probabilidad complementaria es:

$$\begin{aligned} P(S < s) &= 1 - P(S \geq s) \\ P(S \geq s) &= 1 - P(S < s) \end{aligned}$$

Por tanto:

$$\begin{aligned} P(S \geq e^{-\frac{r^2}{2}}) &= 1 - P(S < e^{-\frac{r^2}{2}}) \\ &= 1 - e^{-\frac{r^2}{2}} \end{aligned}$$

Ahora, la probabilidad que  $R'$  esté entre  $r$  y  $r + dr$  es la derivada de  $1 - e^{-\frac{r^2}{2}}$  que es igual a  $re^{-\frac{r^2}{2}} dr$ . Similarmente la probabilidad de que  $\theta'$  esté entre  $\theta$  y  $\theta + d\theta$  es  $\frac{1}{2\pi} d\theta$ . De esta forma, la probabilidad conjunta de  $X_1 \leq x_1$  y  $X_2 \leq x_2$  es:

$$\begin{aligned} \int_{\{(r, \theta) | r \cos \theta \leq x_1, r \sin \theta \leq x_2\}} \frac{1}{2\pi} e^{-\frac{r^2}{2}} r dr d\theta &= \frac{1}{2\pi} \int_{\{(x, y) | x \leq x_1, y \leq x_2\}} e^{-\frac{(x^2 + y^2)}{2}} dx dy \\ &= \left( \sqrt{\frac{1}{2\pi}} \int_{-\infty}^{x_1} e^{-\frac{x^2}{2}} dx \right) \left( \sqrt{\frac{1}{2\pi}} \int_{-\infty}^{x_2} e^{-\frac{y^2}{2}} dy \right) \end{aligned}$$

Lo cual prueba que  $X_1$  y  $X_2$  son variables aleatorias independientes con distribución normal, tal como se requería. □

Basado en el algoritmo 3, podemos derivar un método que permita generar puntos aleatorios dentro de una esfera en 3 dimensiones. Sean  $U_1, U_2 \sim \mathcal{U}(0, 1)$  entonces, hacemos  $V_1, V_2$  y  $S$  como:

$$V_1 = 2U_1 - 1, \quad V_2 = 2U_2 - 1, \quad S = V_1^2 + V_2^2$$

Notamos entonces que  $V_1, V_2 \sim \mathcal{U}(-1, 1)$

Por otro lado, sabemos que la esfera  $\mathbb{S}^2$  tiene las siguientes coordenadas:

$$X = \sin(\theta) \cos(\phi), \quad Y = \sin(\theta) \sin(\phi), \quad Z = \cos(\theta)$$

Notamos que  $Z = \cos(\theta)$  tiene un rango de valores de  $[-1, 1]$ , entonces utilizando la transformación  $Z = 2S - 1$  para  $S = V_1^2 + V_2^2 \leq 1$  esto implica que  $Z = \cos(\theta) = 2S - 1 \in [-1, 1]$  Por otro lado;

$$\begin{aligned}
\cos^2(\theta) + \sin^2(\theta) &= 1 \\
\sin^2(\theta) &= 1 - \cos^2(\theta) \\
\sin^2(\theta) &= 1 - Z^2 \\
\sin(\theta) &= \sqrt{1 - Z^2}
\end{aligned}$$

Por tanto:

$$X = \sqrt{1 - Z^2} \cos(\phi), \quad Y = \sqrt{1 - Z^2} \sin(\phi) \quad (1)$$

Por otro lado, notamos que por construcción

$$\begin{aligned}
S &= V_1^2 + V_2^2 \\
\sqrt{S} &= \sqrt{V_1^2 + V_2^2}
\end{aligned}$$

Y además, en el plano  $XY$

$$\begin{aligned}
\cos(\phi) &= \frac{V_1}{\sqrt{V_1^2 + V_2^2}} \\
&= \frac{V_1}{\sqrt{S}}
\end{aligned}$$

Análogamente:  $\sin(\phi) = \frac{V_2}{\sqrt{S}}$   
Sustituyendo en 1

$$\begin{aligned}
X &= \sqrt{1 - Z^2} \frac{V_1}{\sqrt{S}} \\
&= \frac{\sqrt{1 - Z^2}}{\sqrt{S}} V_1 \\
&= \sqrt{\frac{1 - Z^2}{S}} V_1
\end{aligned}$$

Desarrollando  $\sqrt{\frac{1 - Z^2}{S}}$  :

$$\begin{aligned}
\sqrt{\frac{1-Z^2}{S}} &= \sqrt{\frac{1-(2S-1)^2}{S}} \\
&= \sqrt{\frac{1-((2S)^2+(4S)(-1)+1)}{S}} \\
&= \sqrt{\frac{1-(4S^2-4S+1)}{S}} \\
&= \sqrt{\frac{-4S^2+4S}{S}} \\
&= \sqrt{\frac{4S(-S+1)}{S}} \\
&= \sqrt{4(1-S)} \\
&= \sqrt{4}\sqrt{1-S} \\
&= 2\sqrt{1-S}
\end{aligned}$$

Es decir se toma la parte positiva de la raíz cuadrada ya que se están normalizando coordenadas, por otro lado notamos que  $1-S \geq 0$  pues  $S \leq 1$ . Así, por convención hacemos  $\alpha = 2\sqrt{1-S}$ , por tanto  $X = \alpha V_1$  y análogamente,  $Y = \alpha V_2$ . De esta forma la generación de puntos aleatorios en la superficie de la esfera es:

$$(\alpha V_1, \alpha V_2, 2S-1)$$

Donde:

$$\begin{aligned}
X^2 + Y^2 + Z^2 &= (\alpha V_1)^2 + (\alpha V_2)^2 + (2S-1)^2 \\
&= \alpha^2(V_1^2 + V_2^2) + (2S-1)^2 \\
&= S(2\sqrt{1-S})^2 + (2S-1)^2 \\
&= 4S(1-S) + 4S^2 - 4S + 1 \\
&= -4S^2 + 4S + 4S^2 - 4S + 1 \\
&= 1
\end{aligned}$$

Por último, para generar puntos aleatorios, dentro del volumen de la esfera, basta con generar un radio aleatorio de la siguiente forma: Dado  $R = \mathcal{U}(0, 1)^{\frac{1}{3}}$ , los puntos distribuidos dentro de la esfera están dados por:

$$R(\alpha V_1, \alpha V_2, 2S-1)$$

De esta forma mostraremos el algoritmo resultante en la siguiente forma;



---

**Algorithm 6** Generar Puntos Uniformes en una Esfera

---

**Require:** Coordenadas del centro  $h, k, l$ , número de puntos  $num\_puntos$

**Ensure:** Matriz de puntos uniformemente distribuidos en la esfera

---

```
1: function GENERARPUNTOSUNIFORMES( $h, k, l, num\_puntos$ )
2:   Inicializar una lista vacía  $puntos$ 
3:   for  $i \leftarrow 1$  to  $num\_puntos$  do
4:     repeat
5:       Generar  $V1, V2 \leftarrow \text{UNIFORM}(-1, 1)$ 
6:       Calcular  $S \leftarrow V1^2 + V2^2$ 
7:     until  $S < 1$ 
8:     Calcular  $a \leftarrow \sqrt{1 - S}$ 
9:     Calcular  $x \leftarrow a \cdot V1 + h$ 
10:    Calcular  $y \leftarrow a \cdot V2 + k$ 
11:    Calcular  $z \leftarrow 2 \cdot S - 1 + l$ 
12:    Generar  $r \leftarrow \text{UNIFORM}(0, 1)^{\frac{1}{3}}$ 
13:    Agregar  $[r \cdot x, r \cdot y, r \cdot z]$  a la lista  $puntos$ 
14:  end for
15:  return Matriz convertida a partir de  $puntos$ 
16: end function
```

---

#### 4.5. Reasignación de Adyacencias para Preservar la Estructura

En el punto anterior notamos que cuando se elimina un nodo contenido dentro de la esfera de muestreo, los vecinos de este nodo pierden cierta información estructural contenida en el nodo eliminado, para evitar esto y preservar la estructura lo más parecida posible a la inicial hacemos una reasignación de adyacencias.

Describiremos a continuación el proceso para eliminar un nodo  $v$  de  $G$  y que sus vecinos mantengan la adyacencia entre ellos, es decir si dos nodos  $u, w$  eran vecinos de  $v$  entonces después de eliminar  $v$ ,  $u$  y  $w$  pasaran a ser vecinos entre ellos.

**Definición 52.** Sea  $G$  una gráfica y  $A$  su matriz de adyacencia, entonces definimos al elemento  $(A)_{ij}$  como el elemento  $ij$ -ésimo de la matriz  $A$ , donde  $i, j \in \{0, \dots, \nu_G - 1\}$

**Ejemplo 3.** Sea la matriz  $A$  dada por :

$$A(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix} \quad (2)$$

Entonces los elementos  $(A)_{02}, (A)_{23}, (A)_{54}$  son iguales a :

$$(A)_{02} = 0$$

$$(A)_{23} = 1$$

$$(A)_{54} = 1$$

Dada esta definición podemos ahora establecer una operación que sirve como punto de partida para implementar un método de eliminación controlada para los nodos de la gráfica. La siguiente definición implementará una operación entre los elementos de la matriz de adyacencia  $A_n$  donde  $n$  representa el paso antes de realizar la operación y por ende la matriz  $A_{n+1}$  denota a la matriz de adyacencia  $A$  después de la operación.

**Definición 53** (Eliminación del nodo  $k$ ). *Sea  $G$  una gráfica  $A_n$  la matriz de adyacencia en el paso  $n$ ,  $A_{n+1}$  la matriz de adyacencia en el paso  $n + 1$ ,  $k$  el nodo a eliminar y  $N_G(k)$  el vecindario del nodo  $k$ .*

Entonces definimos la función,

$$\psi : (A_n)_{ij} \times (A_n)_{kj} \rightarrow (A_{n+1})_{ij}$$

Dada por:

$$\psi_{ij} := (A_{n+1})_{ij} = \begin{cases} (A_n)_{ij} +_{or} (A_n)_{kj} & , \quad i \neq j, j \neq k \\ 0 & , \quad k = j, i = j \end{cases} \quad (3)$$

Con  $i \in N_G(k) \cup \{k\}$ , además  $\psi_{kj} = 0, \forall j \in \{0, \dots, \nu_G\}$

**Ejemplo 4.** Dada la matriz  $A_0 = A(G)$  del ejemplo (2),  $k = 5$ ,  $\nu_G = 6$ ,  $j \in \{0, \dots, 5\}$  entonces calcularemos los valores de  $\psi$ .

Observamos que  $N_G(k) = \{1, 2, 4\}$ , entonces:

$$\begin{aligned} \psi_{10} &:= (A_1)_{10} = (A_0)_{10} +_{or} (A_0)_{50} \\ &= 1 +_{or} 0 \\ &= 1 \end{aligned}$$

$$\psi_{11} := (A_1)_{11} = 0$$

$$\begin{aligned} \psi_{12} &:= (A_1)_{12} = (A_0)_{12} +_{or} (A_0)_{52} \\ &= 1 +_{or} 1 \\ &= 1 \end{aligned}$$

$$\psi_{5j} := (A_1)_{5j} = 0 \quad \forall j \in \{0, \dots, 5\}$$

Y notamos que análogamente para  $i = \{2, 4\}$ . De esta forma la matriz  $A_1$  quedaría de la siguiente forma:

$$A_1(G) = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad (4)$$

Así, hemos definido una operación para las matrices de adyacencia donde, dado un nodo  $k$  podemos eliminarlo de dicha matriz, es decir,  $(A)_{kj}, (A)_{ik} = 0$ , donde  $i, j \in \{0, \dots, \nu_G\}$  asegurando que sus vecinos preserven la adyacencia entre ellos, sin modificar la estructura original de la gráfica.

De esta forma podemos definir el algoritmo de la siguiente manera:

---

**Algorithm 7** Eliminar Nodo con Redistribución de Adyacencias

---

**Require:** Nodo a eliminar  $nodo$ , matriz de adyacencia  $matrizAdj$

**Ensure:** Matriz de adyacencia actualizada sin el nodo y con conexiones redistribuidas

---

```

1: function ELIMINANODOCONADYACENCIA( $nodo, matrizAdj$ )
2:   Obtener los vecinos del nodo:  $neighbors \leftarrow \text{WHERE}(matrizAdj[nodo] = 1)$   $\triangleright$  Lista
   de nodos directamente conectados a  $nodo$ .
3:   for all  $v$  en  $neighbors$  do
4:      $matrizAdj[v] \leftarrow \text{LOGICALOR}(matrizAdj[v], matrizAdj[nodo])$   $\triangleright$  Redistribuir
   conexiones del nodo eliminado a sus vecinos.
5:   end for
6:    $matrizAdj[nodo, :] \leftarrow 0$   $\triangleright$  Eliminar todas las conexiones salientes del nodo.
7:    $matrizAdj[:, nodo] \leftarrow 0$   $\triangleright$  Eliminar todas las conexiones entrantes al nodo.
8:   return  $matrizAdj$ 
9: end function

```

---

## 4.6. Método general para la reducción de nodos

Una vez establecidos los métodos descritos anteriormente, es posible combinar sus principios para desarrollar un enfoque generalizado de reducción de nodos.

En esta sección, exploraremos en detalle dicho enfoque, examinando su complejidad computacional, describiendo su implementación paso a paso y destacando las ventajas clave que se derivan de su aplicación.

### 4.6.1. Combinación de funciones

El proceso de muestreo eficiente en una gráfica se basa en reducir el número de nodos mientras se conservan sus propiedades clave. Primero, se genera un orden de recorrido utilizando un árbol generador mínimo y el algoritmo DFS, lo que permite recorrer toda la gráfica y establecer un orden basado en DFS. En el ciclo principal, se iteran los nodos hasta alcanzar el tamaño deseado de la muestra. Se calculan las distancias promedio entre los nodos ordenados y el centroide de la muestra, priorizando los más lejanos para evitar

sesgos. Con el nuevo orden, se generan puntos alrededor de cada nodo utilizando una esfera, identificando y eliminando vecinos dentro de un radio definido, manteniendo la adyacencia. Este proceso se repite hasta obtener la muestra requerida. El algoritmo se divide en fases para facilitar su comprensión.

---

**Algorithm 8** Método General

---

**Require:** Lista de nodos con coordenadas  $x$ , Matriz de adyacencia  $adjacency$ , Tamaño de muestra  $size$ , Radio  $alcance$ , Tamaño de la muestra de esfera  $esfera\_size$

**Ensure:** Nueva lista de nodos  $new\_x$ , Matriz de adyacencia Actualizada  $new\_adj$

---

```

1: function SAMPLING( $x, adjacency, size, alcance, esfera\_size$ )
2:   Obtiene el árbol generador mínimo (MST) desde  $adjacency$   ▷ Se obtiene mediante
   el algoritmo 3
3:   Obtiene el orden DFS en MST
4:   Inicializar  $visited \leftarrow 0$  para todos los nodos
5:   Inicializar  $order \leftarrow []$ 
6:   DFS(0,  $visited$ ,  $order$ )  ▷ Empieza desde el nodo 0
7:   Inicializar  $new\_adj \leftarrow adjacency$ 
8:   Inicializar  $delel \leftarrow []$ 
9:   Inicializar  $queue \leftarrow order$ 
10:   $M, N \leftarrow$  dimensiones de  $adjacency$ 
11:  while  $M - |\text{unique}(delel)| \geq size$  do
12:    if  $delel$  no es vacío then
13:       $selected\_coords \leftarrow x[\text{nodos no están en } delel]$ 
14:      Calcula distancias de  $queue$  nodos al centroide de  $selected\_coords$ 
15:      Ordena  $queue$  por distancias en orden descendente
16:    end if
17:     $idx \leftarrow \text{POP}(queue)$   ▷ Selecciona el siguiente nodo
18:    GENERAPUNTOSUNIFORMES( $idx$ )
19:    Identifica los vecinos dentro de  $alcance$ 
20:     $neighbors \leftarrow$  Intersección de la esfera con los vecinos de  $idx$ 
21:    Actualiza  $queue \leftarrow queue \setminus neighbors$ 
22:    for all  $n \in neighbors \setminus delel$  do
23:      ELIMINANODOCONADYACENCIA( $n, adjacency$ )
24:    end for
25:  end while
26:  return  $new\_x, new\_adj$ 
27: end function

```

---

## 4.7. Análisis de Complejidad

El análisis de complejidad de nuestro algoritmo se realiza descomponiendo sus principales componentes y justificando cada cálculo. La construcción del Árbol Generador Mínimo (MST) utiliza el algoritmo de Kruskal, que tiene una complejidad de  $\mathcal{O}(E \log(V))$  [1], donde  $E$  es el número de aristas y  $V$  el número de vértices del grafo. Este paso es esencial para establecer una estructura inicial que guía el muestreo de nodos. El siguiente paso es ordenar

los nodos del grafo mediante un recorrido en profundidad (DFS), cuya complejidad es  $\mathcal{O}(V + E)$  [1]. Esto se debe a que el DFS visita cada vértice y cada arista exactamente una vez. Este orden nos permite recorrer sistemáticamente los nodos y controlar las operaciones de muestreo.

El bucle principal del algoritmo, que se ejecuta hasta alcanzar el tamaño deseado de la muestra, domina la mayor parte del costo computacional. Dentro de este bucle, el cálculo de distancias entre los nodos y el centroide de la muestra tiene un costo de  $\mathcal{O}(n \cdot d)$ . Este cálculo involucra tres pasos: el cálculo de la media, la resta elemento a elemento y la norma euclidiana para cada nodo en `queue`. Cada paso depende de la dimensión  $d$  de los datos y del número de nodos procesados en esa iteración. La generación de puntos esféricos, que depende únicamente del parámetro `esfera_size`, tiene un costo constante de  $\mathcal{O}(\text{esfera\_size})$ . Como esta operación se repite para cada nodo en el muestreo, el costo total asociado a este componente es  $\mathcal{O}(M \cdot \text{esfera\_size} \cdot d)$ , donde  $M$  es el número de nodos y  $d$  es la dimensión. La intersección de los vecinos generados en la esfera con los vecinos del nodo en la matriz de adyacencia tiene un costo lineal de  $\mathcal{O}(V)$ . Este cálculo depende del tamaño de los conjuntos comparados, donde el conjunto mayor está determinado por el número de vértices del grafo. Una vez identificados los nodos dentro de la esfera, el algoritmo actualiza la matriz de adyacencia. Dado que esta matriz tiene un tamaño de  $V \times V$ , las operaciones de reajuste tienen un costo de  $\mathcal{O}(V^2)$ . Finalmente, los nodos eliminados deben ser descartados de la matriz de adyacencia, lo que implica un costo adicional de  $\mathcal{O}(V \cdot k)$ , donde  $k$  es el número de nodos eliminados. Este costo se suma al procesamiento necesario para mantener la consistencia de la estructura de datos. Resumiendo, notamos que las operaciones más costosas que otorgan la complejidad final al algoritmo es la obtención del MST  $\mathcal{O}(E \log V)$  y las iteraciones del bucle principal, que resulta ser  $\mathcal{O}(M * V^2)$  donde  $T$  depende del tamaño final de la muestra, entonces dado las dos operaciones mas costosas en diferentes fases del algoritmo la complejidad final está dada por:  $\mathcal{O}(E \log V + M * V^2)$ .

## 5. Resultados y conclusiones

Las arquitecturas comúnmente utilizadas en el procesamiento de imágenes requieren que los datos se encuentren ordenados en cierto formato, el cual puede ser en cuadrículas 3D, lo que permite que el proceso de distribución de peso se realice para optimizar el núcleo. Sin embargo, este enfoque puede resultar altamente costoso, ya que maneja una gran cantidad de parámetros que, en muchos casos, no aportan información relevante a la red y generan un desperdicio computacional significativo. Para evitar estos inconvenientes, surge una arquitectura denominada PointNet, diseñada específicamente para procesar directamente nubes de puntos 3D sin la necesidad de transformarlas a una estructura regular como cuadrículas o vóxeles. PointNet aprovecha la naturaleza no estructurada de las nubes de puntos y utiliza operaciones simétricas, como el max pooling, para garantizar la invariancia al orden de los puntos. De este modo, permite realizar el procesamiento de datos 3D de manera más eficiente, capturando tanto características globales como locales directamente de los puntos de entrada, lo que simplifica y optimiza el análisis de formas y estructuras tridimensionales. [7] En el presente capítulo analizaremos la forma en que el algoritmo presentado en el capítulo anterior influye en los resultados de aplicar la arquitectura PointNet para la clasificación de nudos de Lissajous basado en la característica de Euler. La descripción de la arquitectura

está basada en el artículo [7].

## 5.1. Arquitectura PointNet

La arquitectura, está diseñada de tal forma que recibe un conjunto de puntos sin ordenen 3 dimensiones , es decir,  $\{P_i | i = 1, \dots, n\}$  donde cada punto  $P_i$  es un vector con coordenadas  $(x_i, y_i, z_i)$ , es decir, una matriz  $X \in \mathbb{R}^{n \times 3}$  la cual evita la pérdida de información y reduce la complejidad computacional, a diferencia de otros algoritmos que convierten las nubes de puntos en imágenes 2D.

La entrada de la red se basa en tres propiedades principales de los puntos en el espacio Euclidean:

1. **Sin orden:** La nube de puntos es un conjunto de puntos sin un orden en específico es decir la red consume  $N$  puntos en 3d que son invariantes a  $N!$  permutaciones en el conjunto de datos de entrada.
2. **Interacción entre puntos:** Los puntos provienen de un espacio métrico, lo cual otorga un subconjunto significativo de características por lo cual el modelo de poder capturar estructuras locales entre puntos cercanos.
3. **Invariancia bajo transformaciones:** La nube de puntos es invariante bajo ciertas transformaciones por ejemplo, rotar y trasladar el conjunto de puntos no debería modificar la estructura global de dicho conjunto.

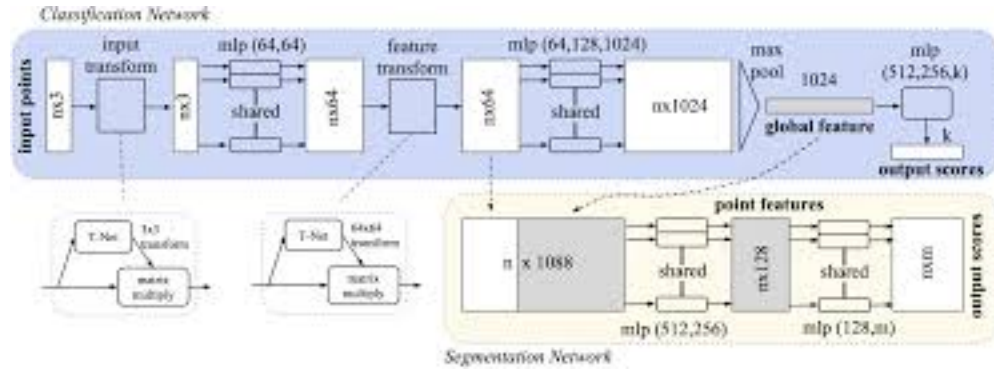


Figura 7: Arquitectura PointNet [7]

En la medida en que el modelo debe permanecer invariante bajo permutaciones de los puntos de entrada, PointNet utiliza una función simétrica para agregar características de puntos individuales. Esta función simétrica es típicamente una operación de max pooling, que selecciona el valor máximo a lo largo de una dimensión específica. La red se compone de varias capas de convolución 1D que operan sobre cada punto individualmente, seguidas de una capa de max pooling global que agrega las características extraídas de todos los puntos.

La arquitectura de PointNet se puede dividir en tres módulos principales:

1. **Transformación de entrada:** PointNet aprende una transformación afín para alinear la nube de puntos de manera canónica, lo cual se logra mediante una subred llamada

**T-Net** la cual predice una matriz de transformación  $T \in \mathbf{R}^{3 \times 3}$ , es decir

$$X^* = X \cdot T$$

, donde  $X^*$  es la nube de puntos alineada y  $T$  es la matriz predicha por **T-Net**

2. **Extracción de características locales y globales:** A través de una red MLP, se extraen características locales de cada punto, es decir, para cada punto  $x_i \in \mathbb{R}^3$ , el MLP lo mapea a un espacio de características de mayor dimensión:

$$h_i = MLP(x_i), \quad h_i \in \mathbb{R}^d$$

lo cual, al mapearse en todos los puntos, resulta en una matriz de características  $H \in \mathbb{R}^{n \times d}$ . Similar a la transformación de entrada, la red utiliza otra **T-Net** para aprender una transformación afín en el espacio de características, es decir:

$$H^* = H \cdot T_f$$

donde  $T_f \in \mathbb{R}^{d \times d}$  es la matriz de transformación aprendida. Luego, una capa de max pooling global agrega estas características en un vector de características globales  $g \in \mathbb{R}^d$ , es decir

$$g_j = \max_{i=1, \dots, n} H_{ij} \quad \forall j = 1, \dots, d$$

3. **Clasificación o segmentación:** Dependiendo de la tarea, el vector de características globales se puede utilizar para la clasificación de la nube de puntos completa, o se pueden concatenar características locales y globales para realizar segmentación punto a punto. En el caso de la clasificación, podemos utilizar una red MLP para predecir la clase, es decir

$$y = MLP(g)$$

donde  $y \in \mathbb{R}^k$  con  $k$  clases y para el vector global  $g$ .

## 5.2. Resultados de la evaluación

## 5.3. Conclusiones

## Referencias

- [1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 3rd edition, 2009.
- [2] Reinhard Diestel. *Graph Theory*. Springer, Graduate Texts in Mathematics, 2017.
- [3] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [4] Tero Harju. *Graph Theory*. Department of Mathematics University of Turku, 2007.
- [5] D Knuth. *The Art of Computer Programming*. ADDISON-WESLEY An Imprint of Addison Wesley Longman, Inc., 1998.

- [6] Andrew Pressley. *Elementary Differential Geometry*. Springer, Undergraduate Texts in Mathematics, 2000.
- [7] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation. *arXiv preprint*, arXiv:1612.00593, 2017.